```
# Name : Nitin Jain Roll No : M24AID022  Sub: Assignment 3 fo ML for Big data
# Recommender Systems: Content-Based and Collaborative Filtering
# Github Repository LinK : https://github.com/nitinjain2015/M24AID022_CSL7110_Assignment3

!pip install pandas numpy scikit-learn scipy surprise tqdm shap lime
```

```
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (1.16.3)
Requirement already satisfied: surprise in /usr/local/lib/python3.12/dist-packages (0.1)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (4.67.3)
Requirement already satisfied: shap in /usr/local/lib/python3.12/dist-packages (0.51.0)
Requirement already satisfied: lime in /usr/local/lib/python3.12/dist-packages (0.2.0.1)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: scikit-surprise in /usr/local/lib/python3.12/dist-packages (from surprise) (1.1.4)
Requirement already satisfied: packaging>20.9 in /usr/local/lib/python3.12/dist-packages (from shap) (26.0)
Requirement already satisfied: slicer==0.0.8 in /usr/local/lib/python3.12/dist-packages (from shap) (0.0.8)
Requirement already satisfied: numba in /usr/local/lib/python3.12/dist-packages (from shap) (0.60.0)
Requirement already satisfied: llvmlite in /usr/local/lib/python3.12/dist-packages (from shap) (0.43.0)
Requirement already satisfied: cloudpickle in /usr/local/lib/python3.12/dist-packages (from shap) (3.1.2)
Requirement already satisfied: typing-extensions in /usr/local/lib/python3.12/dist-packages (from shap) (4.15.0)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (from lime) (3.10.0)
Requirement already satisfied: scikit-image>=0.12 in /usr/local/lib/python3.12/dist-packages (from lime) (0.25.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas) (1.
Requirement already satisfied: networkx>=3.0 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (3.6
Requirement already satisfied: pillow>=10.1 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (11.3
Requirement already satisfied: imageio!=2.35.0,>=2.33 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->l
Requirement already satisfied: tifffile>=2022.8.12 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime
Requirement already satisfied: lazy-loader>=0.4 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (4.62.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (1.5.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (3.3.2)
```

```
# Import the libraries
import pandas as pd
import numpy as np

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.metrics import mean_squared_error

from sklearn.model_selection import train_test_split

from scipy.sparse.linalg import svds

import matplotlib.pyplot as plt
import seaborn as sns

from tqdm import tqdm
```

Task 1: Implementing TF-IDF Based Recommendation In this task, you will build a content-based recommender system using Term Frequency-Inverse Document Frequency (TF IDF). The goal is to recommend movies similar to a given movie based on textual features.

```
# Download the Movie Lens dataset small

!wget https://files.grouplens.org/datasets/movielens/ml-latest-small.zip
!unzip -o ml-latest-small.zip
```

```
--2026-03-18 13:34:59--  https://files.grouplens.org/datasets/movielens/ml-latest-small.zip
Resolving files.grouplens.org (files.grouplens.org)... 128.101.96.204
Connecting to files.grouplens.org (files.grouplens.org)|128.101.96.204|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 978202 (955K) [application/zip]
Saving to: ‘ml-latest-small.zip.1’

ml-latest-small.zip 100%[===================>] 955.28K   906KB/s    in 1.1s

2026-03-18 13:35:01 (906 KB/s) - ‘ml-latest-small.zip.1’ saved [978202/978202]

Archive:  ml-latest-small.zip
  inflating: ml-latest-small/links.csv
  inflating: ml-latest-small/tags.csv
  inflating: ml-latest-small/ratings.csv
  inflating: ml-latest-small/README.txt
```

```
  inflating: ml-latest-small/movies.csv
```

```
# Load the dataset

movies = pd.read_csv('ml-latest-small/movies.csv')
ratings = pd.read_csv('ml-latest-small/ratings.csv')
tags = pd.read_csv('ml-latest-small/tags.csv')
links = pd.read_csv('ml-latest-small/links.csv')

print("Movies:", movies.shape)
print("Ratings:", ratings.shape)
print("Tags:", tags.shape)
print("Links:", links.shape)

movies.head()
# ratings.head()
# tags.head()
# links.head()
```

```
Movies: (9742, 3)
Ratings: (100836, 4)
Tags: (3683, 4)
Links: (9742, 3)
```

	movieId	title	genres
0	1	Toy Story (1995)	Adventure|Animation|Children|Comedy|Fantasy
1	2	Jumanji (1995)	Adventure|Children|Fantasy
2	3	Grumpier Old Men (1995)	Comedy|Romance
3	4	Waiting to Exhale (1995)	Comedy|Drama|Romance
4	5	Father of the Bride Part II (1995)	Comedy

```
# Extract the Movie Genre - replace every occurrence of the pipe character (|) with a space (' ').

movies['genres'] = movies['genres'].str.replace('|',' ')

movies[['movieId','title','genres']].head()
```

	movieId	title	genres
0	1	Toy Story (1995)	Adventure Animation Children Comedy Fantasy
1	2	Jumanji (1995)	Adventure Children Fantasy
2	3	Grumpier Old Men (1995)	Comedy Romance
3	4	Waiting to Exhale (1995)	Comedy Drama Romance
4	5	Father of the Bride Part II (1995)	Comedy

```
# Compute TF IDF

tfidf = TfidfVectorizer(stop_words='english')

tfidf_matrix = tfidf.fit_transform(movies['genres'])

tfidf_matrix.shape
tfidf_matrix[:5].toarray()
```

```
array([[0.        , 0.41684567, 0.51622547, 0.50484547, 0.26758648,
        0.        , 0.        , 0.        , 0.48299014, 0.        ,
        0.        , 0.        , 0.        , 0.        , 0.        ,
        0.        , 0.        , 0.        , 0.        , 0.        ,
        0.        , 0.        , 0.        ],
       [0.        , 0.51236121, 0.        , 0.62052517, 0.        ,
        0.        , 0.        , 0.        , 0.59366194, 0.        ,
        0.        , 0.        , 0.        , 0.        , 0.        ,
        0.        , 0.        , 0.        , 0.        , 0.        ,
        0.        , 0.        , 0.        ],
       [0.        , 0.        , 0.        , 0.        , 0.57091541,
        0.        , 0.        , 0.        , 0.        , 0.        ,
        0.        , 0.        , 0.        , 0.        , 0.        ,
        0.        , 0.        , 0.        , 0.82100889, 0.        ,
        0.        , 0.        , 0.        ],
       [0.        , 0.        , 0.        , 0.        , 0.50501544,
        0.        , 0.        , 0.4664048 , 0.        , 0.        ,
        0.        , 0.        , 0.        , 0.        , 0.        ,
        0.        , 0.        , 0.        , 0.72624098, 0.        ,
        0.        , 0.        , 0.        ],
       [0.        , 0.        , 0.        , 0.        , 1.        ,
```

```
        0.        , 0.        , 0.        , 0.        , 0.        ,
        0.        , 0.        , 0.        , 0.        , 0.        ,
        0.        , 0.        , 0.        , 0.        , 0.        ,
        0.        , 0.        , 0.        ]])
```

```
# Find cosine similarity

cosine_sim = cosine_similarity(tfidf_matrix, tfidf_matrix)

cosine_sim.shape
cosine_sim[:5]
```

```
array([[1.        , 0.81357774, 0.15276924, ..., 0.        , 0.4210373 ,
        0.26758648],
       [0.81357774, 1.        , 0.        , ..., 0.        , 0.        ,
        0.        ],
       [0.15276924, 0.        , 1.        , ..., 0.        , 0.        ,
        0.57091541],
       [0.1351353 , 0.        , 0.8845714 , ..., 0.4664048 , 0.        ,
        0.50501544],
       [0.26758648, 0.        , 0.57091541, ..., 0.        , 0.        ,
        1.        ]])
```

```
# Map Movie Index

indices = pd.Series(movies.index, index=movies['title']).drop_duplicates()

indices.head()
```

title	0
Toy Story (1995)	0
Jumanji (1995)	1
Grumpier Old Men (1995)	2
Waiting to Exhale (1995)	3
Father of the Bride Part II (1995)	4

dtype: int64

```
# Building the Rcommender Function

def recommend_movies(title, top_n=5):

    idx = indices[title]

    sim_scores = list(enumerate(cosine_sim[idx]))

    sim_scores = sorted(sim_scores, key=lambda x: x[1], reverse=True)

    sim_scores = sim_scores[1:top_n+1]

    movie_indices = [i[0] for i in sim_scores]

    scores = [i[1] for i in sim_scores]

    result = movies.iloc[movie_indices][['title']]
    result['similarity'] = scores

    return result
```

```
# Recommended Movies : Input a few movie titles and print the top-5 recommended movies along with their cosine similarity s

print("Recommendations for Movie : Toy Story (1995)")
display(recommend_movies("Toy Story (1995)",5))

print("\nRecommendations for Movie : Jumanji (1995)")
display(recommend_movies("Jumanji (1995)",5))

print("\nRecommendations for Movie : Grumpier Old Men (1995)")
display(recommend_movies("Grumpier Old Men (1995)",5))
```

```
Recommendations for Movie : Toy Story (1995)
```

	title	similarity
1706	Antz (1998)	1.0
2355	Toy Story 2 (1999)	1.0
2809	Adventures of Rocky and Bullwinkle, The (2000)	1.0
3000	Emperor's New Groove, The (2000)	1.0
3568	Monsters, Inc. (2001)	1.0

```
Recommendations for Movie : Jumanji (1995)
```

	title	similarity
53	Indian in the Cupboard, The (1995)	1.0
109	NeverEnding Story III, The (1994)	1.0
767	Escape to Witch Mountain (1975)	1.0
1514	Darby O'Gill and the Little People (1959)	1.0
1556	Return to Oz (1985)	1.0

```
Recommendations for Movie : Grumpier Old Men (1995)
```

	title	similarity
6	Sabrina (1995)	1.0
35	Clueless (1995)	1.0
57	Two if by Sea (1996)	1.0
60	French Twist (Gazon maudit) (1995)	1.0
103	If Lucy Fell (1996)	1.0

Task 2: User-Profile-Based Content Recommender In this task, you will build a content-based recommender system that personalizes recommendations based on user preferences. Instead of relying solely on item descriptions, you will construct user profiles from their historical interactions (i.e., rated movies) and use them to suggest relevant movies.

```
import pandas as pd
data = pd.merge(ratings,movies,on='movieId')

data.head()
```

Show hidden output

```
# Build User Profile

def build_user_profile(user_id):

    # Get movies rated by the user
    user_ratings = ratings[ratings['userId'] == user_id]

    # Get indices of these movies in movies dataframe
    movie_indices = movies[movies['movieId'].isin(user_ratings['movieId'])].index

    # Corresponding TF-IDF vectors
    movie_vectors = tfidf_matrix[movie_indices]

    # Ratings as weights
    ratings_vector = user_ratings['rating'].values.reshape(-1,1)

    # Weighted sum of movie vectors
    weighted_sum = movie_vectors.multiply(ratings_vector).sum(axis=0)

    # Normalize by total rating
    user_profile = weighted_sum / ratings_vector.sum()

    return np.asarray(user_profile)
```

```
profile = build_user_profile(1)

print("User profile shape:", profile.shape)
```

```
User profile shape: (1, 23)
```

```
# Recommend Movies based on user profile

def recommend_for_user(user_id, top_n=5):

    user_profile = build_user_profile(user_id)

    scores = cosine_similarity(user_profile, tfidf_matrix).flatten()

    # Movies already rated by user
    rated_movies = ratings[ratings['userId']==user_id]['movieId']

    # Remove them
    candidate_movies = movies[~movies['movieId'].isin(rated_movies)]

    candidate_indices = candidate_movies.index

    candidate_scores = scores[candidate_indices]

    top_indices = candidate_indices[np.argsort(candidate_scores)[::-1][:top_n]]

    return movies.iloc[top_indices][['movieId','title']]
```

```
recommend_for_user(1,5)
```

	movieId	title
8597	117646	Dragonheart 2: A New Beginning (2000)
6570	55116	Hunting Party, The (2007)
4681	6990	The Great Train Robbery (1978)
4005	5657	Flashback (1990)
4409	6503	Charlie's Angels: Full Throttle (2003)

```
# Evaluation function

def evaluate_user(user_id, K=5):

    liked_movies = data[(data['userId']==user_id) & (data['rating']>=4)]
    ground_truth = set(liked_movies['movieId'])

    recommended = recommend_for_user(user_id, K)

    recommended_ids = set(recommended['movieId'])

    relevant = recommended_ids.intersection(ground_truth)

    precision = len(relevant) / K
    recall = len(relevant) / len(ground_truth)

    return precision, recall
```

```
precision, recall = evaluate_user(1,5)

print("Precision@5:", precision)
print("Recall@5:", recall)

# Incase The precision and recall for user are 0, it implies none of the recommended movies appear in the user's highly rat
# the content-based recommender uses only genre features, which may not fully capture user preferences.
```

```
Precision@5: 0.0
Recall@5: 0.0
```

**Conclusion Rask 2 Evaluation **: The precision and recall for user 1 are 0, it implies none of the recommended movies appear in the user's highly rated movies. This is because the content-based recommender uses only genre features, which may not fully capture user preferences.

Part 2: Collaborative Filtering

Task 3: User-Based Collaborative Filtering

User-based collaborative filtering recommends items based on the preferences of similar users. The assumption is that users with similar tastes in the past will have similar preferences in the future.

1. Populate a user-movie rating matrix from the dataset

2. Compute user-user similarity Calculate pairwise similarity between users using either: cosine similarity or Pearson correlation ~ Hint: Consider which is a more suitable metric if user ratings have different biases (e.g., some users rate generously while others rate harshly).
3. Predict ratings for a given user using weighted average of neighbors Select the K most similar users. Compute the weighted average of their ratings, giving more weight to users with higher similarity scores. Predict ratings for movies the user has not rated.
4. Implement a recommendation function based on predicted ratings Rank movies based on the predicted ratings. Recommend the top-N highest-rated movies for the target user.
5. Evaluate the system with RMSE (Root Mean Squared Error), Precision@K, and Recall@K
6. Test with different values of K (number of similar users) to see how recommendation quality changes.

```
# Solution Task 3
#  User-Based Collaborative Filtering

import numpy as np
import pandas as pd
from sklearn.metrics.pairwise import cosine_similarity

# Create the User-Item Matrix
user_item = ratings.pivot_table(index='userId',
                                columns='movieId',
                                values='rating')

# Handling NaN (for cosine similarity)
user_item_filled = user_item.fillna(0)

# Compute User Similarity
user_similarity = cosine_similarity(user_item_filled)
user_similarity = pd.DataFrame(user_similarity,
                               index=user_item.index,
                               columns=user_item.index)

# Predicting Rating Function
def predict_rating(user_id, movie_id, k=5):

    # Sellect the k most similar users
    similar_users = user_similarity[user_id].drop(user_id)
    similar_users = similar_users.sort_values(ascending=False).head(k)

    ratings_subset = user_item.loc[similar_users.index, movie_id]

    mask = ratings_subset.notnull()

    if mask.sum() == 0:
        return np.nan

    sim_scores = similar_users[mask]
    ratings_subset = ratings_subset[mask]

    if sim_scores.sum() == 0:
        return np.nan

    # Compute weighted average of the ratings
    return np.dot(sim_scores, ratings_subset) / sim_scores.sum()

#  Creating Recommendation Function
def recommend_cf(user_id, top_n=5):

    # Predicting ratings for movies not rated
    rated_movies = user_item.loc[user_id].dropna().index
    movies_not_rated = user_item.columns.difference(rated_movies)

    predictions = []
    for movie in movies_not_rated:
        pred = predict_rating(user_id, movie)

        if not np.isnan(pred):
            predictions.append((movie, pred))


    # Rank the movies based on predicted ratings and recommend top n highest rated movies
    predictions = sorted(predictions, key=lambda x: x[1], reverse=True)[:top_n]
    movie_ids = [i[0] for i in predictions]
    return movies[movies['movieId'].isin(movie_ids)][['movieId','title']].reset_index(drop=True)

# Print top 5 recommendations for user 1
print("Top 5 Recommendations for User 1:")
print(recommend_cf(1,5))
```

```
Top 5 Recommendations for User 1:
   movieId                                           title
0      514                                   Ref, The (1994)
1      541                                Blade Runner (1982)
2      720  Wallace & Gromit: The Best of Aardman Animatio...
3      750  Dr. Strangelove or: How I Learned to Stop Worr...
4      858                              Godfather, The (1972)
```

```
# Task 3 - Evaluation (RMSE, Precision@K, Recall@K)

from sklearn.metrics import mean_squared_error
import numpy as np

K = 5

actual = []
predicted = []

precisions = []
recalls = []

# Loop through users
for user_id in user_item.index[:50]:   # use 50 users for speed

    # Get movies rated by user
    user_ratings = user_item.loc[user_id].dropna()

    for movie_id in user_ratings.index:

        pred = predict_rating(user_id, movie_id, k=K)

        if not np.isnan(pred):
            actual.append(user_ratings[movie_id])
            predicted.append(pred)

    # Precision & Recall
    liked_movies = data[(data['userId']==user_id) & (data['rating']>=4)]
    ground_truth = set(liked_movies['movieId'])

    recommended = recommend_cf(user_id, K)
    recommended_ids = set(recommended['movieId'])

    relevant = recommended_ids.intersection(ground_truth)

    if len(recommended_ids) > 0:
        precision = len(relevant) / len(recommended_ids)
        recall = len(relevant) / len(ground_truth) if len(ground_truth)>0 else 0

        precisions.append(precision)
        recalls.append(recall)

# RMSE
rmse = np.sqrt(mean_squared_error(actual, predicted))

# Average Precision & Recall
avg_precision = np.mean(precisions)
avg_recall = np.mean(recalls)

print("RMSE:", rmse)
print("Average Precision@5:", avg_precision)
print("Average Recall@5:", avg_recall)
```

```
RMSE: 1.1458395012740212
Average Precision@5: 0.0
Average Recall@5: 0.0
```

```
# Task 3 - Evaluation (RMSE + Precision@K + Recall@K)

from sklearn.metrics import mean_squared_error
import numpy as np

K = 5

actual = []
predicted = []

precisions = []
recalls = []

for user_id in user_item.index[:50]:   # use 50 users

    user_ratings = user_item.loc[user_id].dropna()
```

```
    # RMSE
    for movie_id in user_ratings.index:

        pred = predict_rating(user_id, movie_id, k=K)

        if not np.isnan(pred):
            actual.append(user_ratings[movie_id])
            predicted.append(pred)

    # Precision & Recall
    liked_movies = data[(data['userId']==user_id) & (data['rating']>=4)]
    ground_truth = set(liked_movies['movieId'])

    recommended = recommend_cf(user_id, K)
    recommended_ids = set(recommended['movieId'])

    relevant = recommended_ids.intersection(ground_truth)

    if len(recommended_ids) > 0:
        precision = len(relevant) / len(recommended_ids)
        recall = len(relevant) / len(ground_truth) if len(ground_truth)>0 else 0

        precisions.append(precision)
        recalls.append(recall)

# Final metrics
rmse = np.sqrt(mean_squared_error(actual, predicted)) if len(actual)>0 else np.nan
avg_precision = np.mean(precisions) if len(precisions)>0 else 0
avg_recall = np.mean(recalls) if len(recalls)>0 else 0

print("RMSE:", rmse)
print("Average Precision@5:", avg_precision)
print("Average Recall@5:", avg_recall)
```

```
RMSE: 1.1458395012740212
Average Precision@5: 0.0
Average Recall@5: 0.0
```

Conclusion : Task 3 - Step 5 :RMSE value indicates good prediction accuracy, but Precision and Recall are zero due to lack of overlap between recommended movies and highly rated movies. This is because of data sparsity and the nature of collaborative filtering, which recommends unseen items.

```
# Solution for Task 3- Step 6  Test with different values of K (number of similar users) to see how recommendation quality

from sklearn.metrics import mean_squared_error
import numpy as np
import pandas as pd

K_values = [2, 5, 10, 20]
results = []

for K in K_values:

    actual = []
    predicted = []
    precisions = []
    recalls = []

    for user_id in user_item.index[:30]:   # limit users for speed

        user_ratings = user_item.loc[user_id].dropna()

        # RMSE
        for movie_id in user_ratings.index:
            pred = predict_rating(user_id, movie_id, k=K)
            if not np.isnan(pred):
                actual.append(user_ratings[movie_id])
                predicted.append(pred)

        # Precision & Recall
        liked_movies = data[(data['userId']==user_id) & (data['rating']>=4)]
        ground_truth = set(liked_movies['movieId'])

        recommended = recommend_cf(user_id, 5)
        recommended_ids = set(recommended['movieId'])

        relevant = recommended_ids.intersection(ground_truth)

        if len(recommended_ids) > 0:
```

```
            precision = len(relevant) / len(recommended_ids)
            recall = len(relevant) / len(ground_truth) if len(ground_truth)>0 else 0

            precisions.append(precision)
            recalls.append(recall)

    rmse = np.sqrt(mean_squared_error(actual, predicted)) if len(actual)>0 else np.nan
    avg_precision = np.mean(precisions) if len(precisions)>0 else 0
    avg_recall = np.mean(recalls) if len(recalls)>0 else 0

    results.append([K, rmse, avg_precision, avg_recall])

# Display results
results_df = pd.DataFrame(results, columns=["K", "RMSE", "Precision@5", "Recall@5"])

results_df
```

Conclusion Task 3- Step 6 : As the value of K increases , RMSE decreases. Thus recommendations become more stable but less personalized. Precision and Recall are still zero due to lack of overlap between recommended movies and highly rated movies. This is because of data sparsity and the nature of collaborative filtering, which recommends unseen items.

Task 4: Item-Based Collaborative Filtering Item-Based Collaborative Filtering recommends items based on their similarity to items a user has previously liked or rated highly. The key idea is: If two items are rated similarly by many users, they are likely to be similar.

1. Compute item-item similarity using Pearson correlation or cosine similarity
2. Predict ratings for a given user based on similar items Identify items the user has already rated. Find similar items and use their ratings to predict new ratings. Compute the weighted average of similar item ratings, where the weights are similarity scores. ~ Hint: Higher similarity = higher weight in rating prediction.
3. Generate top-N movie recommendations for a user
4. Evaluate the system and compare the recommendations with user-based CF to analyze differences in output.
5. Do you think item based CF may be faster or more memory-efficient than user based CF in the real world? Why?

```
# Code for Task 4 - Step 1
# Compute Item-Item Similarity

from sklearn.metrics.pairwise import cosine_similarity
import pandas as pd
import numpy as np

# User-Item matrix (already created earlier)
# user_item

# Replace NaN with 0 for similarity
item_matrix = user_item.fillna(0).T   # transpose → items as rows

# Compute similarity
item_similarity = cosine_similarity(item_matrix)

item_similarity = pd.DataFrame(item_similarity,
                               index=item_matrix.index,
                               columns=item_matrix.index)

item_similarity.iloc[:5, :5]
```

```
#Code for task 4- Step 2
# Predict Ratings for a given used based on Item Similarity

def predict_item_rating(user_id, movie_id, k=5):

    # Similar items
    similar_items = item_similarity[movie_id].drop(movie_id)
    similar_items = similar_items.sort_values(ascending=False).head(k)

    # User ratings for similar items
    user_ratings = user_item.loc[user_id, similar_items.index]

    mask = user_ratings.notnull()

    if mask.sum() == 0:
        return np.nan

    sim_scores = similar_items[mask]
    user_ratings = user_ratings[mask]

    if sim_scores.sum() == 0:
        return np.nan
```

```
    return np.dot(sim_scores, user_ratings) / sim_scores.sum()
```

```
#  Generate Top N Movie Recommenderss for a user

def recommend_item_cf(user_id, top_n=5):

    rated_movies = user_item.loc[user_id].dropna().index
    movies_not_rated = user_item.columns.difference(rated_movies)

    predictions = []

    for movie in movies_not_rated:

        pred = predict_item_rating(user_id, movie)

        if not np.isnan(pred):
            predictions.append((movie, pred))

    predictions = sorted(predictions, key=lambda x: x[1], reverse=True)[:top_n]

    movie_ids = [i[0] for i in predictions]

    return movies[movies['movieId'].isin(movie_ids)][['movieId','title']].reset_index(drop=True)
```

```
# Evaluation for Item-Based CF

from sklearn.metrics import mean_squared_error

K = 5

actual = []
predicted = []
precisions = []
recalls = []

for user_id in user_item.index[:30]:

    user_ratings = user_item.loc[user_id].dropna()

    # RMSE
    for movie_id in user_ratings.index:

        pred = predict_item_rating(user_id, movie_id)

        if not np.isnan(pred):
            actual.append(user_ratings[movie_id])
            predicted.append(pred)

    # Precision & Recall
    liked_movies = data[(data['userId']==user_id) & (data['rating']>=4)]
    ground_truth = set(liked_movies['movieId'])

    recommended = recommend_item_cf(user_id, K)
    recommended_ids = set(recommended['movieId'])

    relevant = recommended_ids.intersection(ground_truth)

    if len(recommended_ids) > 0:
        precision = len(relevant) / len(recommended_ids)
        recall = len(relevant) / len(ground_truth) if len(ground_truth)>0 else 0

        precisions.append(precision)
        recalls.append(recall)

rmse = np.sqrt(mean_squared_error(actual, predicted)) if len(actual)>0 else np.nan
avg_precision = np.mean(precisions) if len(precisions)>0 else 0
avg_recall = np.mean(recalls) if len(recalls)>0 else 0

print("RMSE:", rmse)
print("Precision@5:", avg_precision)
print("Recall@5:", avg_recall)
```

Conclusion Task 4 - Step 5

Results show RMSE decreases with Item based Collabative Filtering while Precision and Recall are still 0 on account of sparse data. Item-Based Collaborative Filtering in general is more stable and scalable because item similarity changes less frequently compared to user similarity. Also it is often faster in real-world systems as items are generally fewer and more static compared to users.

Task 5: Implement SVD for Recommendations

```
# Code for Task 5 - Step 1
# Prepare User-Item Matrix for Factorozation

import numpy as np
import pandas as pd

# Create user-item matrix
user_item = ratings.pivot_table(index='userId',
                                columns='movieId',
                                values='rating')

# Fill missing values with 0
R = user_item.fillna(0).values

print("Matrix shape:", R.shape)
```

```
# Code for Task 5 - Step 2: Apply Singular Value Decomposition

from scipy.sparse.linalg import svds

# Choose number of latent factors
k = 20

U, sigma, Vt = svds(R, k=k)

# Convert sigma to diagonal matrix
sigma = np.diag(sigma)

print("U shape:", U.shape)
print("Sigma shape:", sigma.shape)
print("Vt shape:", Vt.shape)
```

```
# Code for Task 5 - Step 3: Apply Singular Value Decomposition

# Reconstruct matrix
predicted_ratings = np.dot(np.dot(U, sigma), Vt)

# Convert to DataFrame
pred_df = pd.DataFrame(predicted_ratings,
                       index=user_item.index,
                       columns=user_item.columns)

pred_df.head()
```

```
# # Contd Code for Task 5 - Step 4: Generate Recommendations for  Movies using SVD

def recommend_svd(user_id, top_n=5):

    user_ratings = user_item.loc[user_id]
    user_predictions = pred_df.loc[user_id]

    # Remove already rated movies
    unrated_movies = user_ratings[user_ratings.isna()].index

    recommendations = user_predictions[unrated_movies]\
                        .sort_values(ascending=False)\
                        .head(top_n)

    return movies[movies['movieId'].isin(recommendations.index)][['movieId','title']].reset_index(drop=True)

# Test
recommend_svd(1,5)
```

```
# Task 5 - Step 5 : Compare SVD vs User-CF vs Item-CF

from sklearn.metrics import mean_squared_error

actual = []
svd_pred = []
usercf_pred = []
itemcf_pred = []

for user_id in user_item.index[:30]:   # limit for speed
```

```
    user_ratings = user_item.loc[user_id].dropna()

    for movie_id in user_ratings.index:

        # Actual
        actual.append(user_ratings[movie_id])

        # SVD prediction
        svd_pred.append(pred_df.loc[user_id, movie_id])

        # User-CF prediction
        u_pred = predict_rating(user_id, movie_id)
        usercf_pred.append(u_pred if not np.isnan(u_pred) else 0)

        # Item-CF prediction
        i_pred = predict_item_rating(user_id, movie_id)
        itemcf_pred.append(i_pred if not np.isnan(i_pred) else 0)

# Compute RMSE
rmse_svd = np.sqrt(mean_squared_error(actual, svd_pred))
rmse_user = np.sqrt(mean_squared_error(actual, usercf_pred))
rmse_item = np.sqrt(mean_squared_error(actual, itemcf_pred))

print("RMSE Comparison:")
print("SVD:", rmse_svd)
print("User-CF:", rmse_user)
print("Item-CF:", rmse_item)
```

Conclusion Task - 5 Step 4 Results show SVD performs better than user-based and item-based collaborative filtering because it captures latent factors representing hidden user preferences and item characteristics.SVD provides more accurate predictions as it reduces sparsity.

Task 6: Implementing Matrix Factorization with the Surprise Librar

```
# Code Task 6 - Step 1 : Loading the dataset into Surprise library  and split

from surprise import Dataset, Reader
from surprise.model_selection import train_test_split

# Define reader (rating scale 1-5)
reader = Reader(rating_scale=(1,5))

# Load dataset
data_surprise = Dataset.load_from_df(ratings[['userId','movieId','rating']], reader)

# Train-test split
trainset, testset = train_test_split(data_surprise, test_size=0.2, random_state=42)

print("Train size:", len(trainset.all_ratings()))
print("Test size:", len(testset))
```

```
#  Code Task 6 - Step 2 : Training the  SVD model

from surprise import SVD

# Initialize model
model = SVD(n_factors=50, lr_all=0.005, reg_all=0.02)

# Train
model.fit(trainset)

print("SVD model trained successfully")
```

```
# Code Tsk 6 -  Step 3 :Evaluate model performance with RMSE, Precision@K, and Recall@K

from surprise import accuracy
import numpy as np
from collections import defaultdict

# Predictions on test set
predictions = model.test(testset)

# RMSE
rmse = accuracy.rmse(predictions)

# Precision@K & Recall@K

def precision_recall_at_k(predictions, k=5, threshold=4):
```

```
    user_est_true = defaultdict(list)

    for uid, iid, true_r, est, _ in predictions:
        user_est_true[uid].append((est, true_r))

    precisions = {}
    recalls = {}

    for uid, user_ratings in user_est_true.items():

        user_ratings.sort(key=lambda x: x[0], reverse=True)

        top_k = user_ratings[:k]

        relevant = sum((true_r >= threshold) for (_, true_r) in user_ratings)
        recommended = sum((est >= threshold) for (est, _) in top_k)
        relevant_and_recommended = sum((true_r >= threshold and est >= threshold) for (est, true_r) in top_k)

        precision = relevant_and_recommended / recommended if recommended > 0 else 0
        recall = relevant_and_recommended / relevant if relevant > 0 else 0

        precisions[uid] = precision
        recalls[uid] = recall

    return np.mean(list(precisions.values())), np.mean(list(recalls.values()))

precision, recall = precision_recall_at_k(predictions, k=5)

print("Precision@5:", precision)
print("Recall@5:", recall)
```

```
# Code Task 6 - Step  4 : Comparison pf Surprise SVDwith normal  SVD in Task 5

print("Comparison of Models:")
print("Surprise SVD RMSE:", rmse)

# print("Manual SVD RMSE:", rmse_svd)

print("\nObservation:")
print("Surprise SVD typically performs better due to optimized learning and hyperparameter tuning.")
```

Task 7: Implementing a Hybrid Recommendation Model

```
# Task 7 - Hybrid Recommendation Model

import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.metrics import mean_squared_error

# Prepare the  Training Data

rows = []

for _, row in ratings.iterrows():

    user = row['userId']
    movie = row['movieId']
    true_rating = row['rating']

    # Calculate CBF Score
    try:
        user_profile = build_user_profile(user)
        movie_idx = movies[movies['movieId']==movie].index

        if len(movie_idx) == 0:
            continue

        cbf_score = cosine_similarity(user_profile, tfidf_matrix[movie_idx]).flatten()[0]
    except:
        continue

    # CF Score
    cf_score = predict_rating(user, movie)

    if np.isnan(cf_score):
        continue
```

```
    # Additional Features
    popularity = ratings[ratings['movieId']==movie]['rating'].mean()
    user_avg = ratings[ratings['userId']==user]['rating'].mean()

    rows.append([cbf_score, cf_score, popularity, user_avg, true_rating])

df_meta = pd.DataFrame(rows, columns=['cbf','cf','popularity','user_avg','rating'])


# Train Meta Model

X = df_meta[['cbf','cf','popularity','user_avg']]
y = df_meta['rating']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

meta_model = RandomForestRegressor(n_estimators=50, random_state=42)
meta_model.fit(X_train, y_train)


#  Hybrid Prediction

def hybrid_predict(user_id, movie_id):

    movie_idx = movies[movies['movieId']==movie_id].index
    if len(movie_idx)==0:
        return np.nan

    # CBF
    user_profile = build_user_profile(user_id)
    cbf_score = cosine_similarity(user_profile, tfidf_matrix[movie_idx]).flatten()[0]

    # CF
    cf_score = predict_rating(user_id, movie_id)
    if np.isnan(cf_score):
        return np.nan

    # Additional features
    popularity = ratings[ratings['movieId']==movie_id]['rating'].mean()
    user_avg = ratings[ratings['userId']==user_id]['rating'].mean()

    features = np.array([[cbf_score, cf_score, popularity, user_avg]])

    return meta_model.predict(features)[0]

#  Recommendation

def recommend_hybrid(user_id, top_n=5):

    rated_movies = user_item.loc[user_id].dropna().index
    candidate_movies = user_item.columns.difference(rated_movies)

    preds = []

    for movie in candidate_movies:

        pred = hybrid_predict(user_id, movie)

        if not np.isnan(pred):
            preds.append((movie, pred))

    preds = sorted(preds, key=lambda x: x[1], reverse=True)[:top_n]

    movie_ids = [i[0] for i in preds]

    return movies[movies['movieId'].isin(movie_ids)][['movieId','title']].reset_index(drop=True)


#Evaluation (RMSE)


actual = []
predicted = []

for user_id in user_item.index[:30]:

    user_ratings = user_item.loc[user_id].dropna()

    for movie_id in user_ratings.index:

        pred = hybrid_predict(user_id, movie_id)
```

```
        if not np.isnan(pred):
            actual.append(user_ratings[movie_id])
            predicted.append(pred)

rmse_hybrid = np.sqrt(mean_squared_error(actual, predicted)) if len(actual)>0 else np.nan

print("Hybrid Model RMSE:", rmse_hybrid)

# Test Recommendation

print("\nTop 5 Hybrid Recommendations for User 1:")
print(recommend_hybrid(1,5))
```

Conclusion Q7 A hybrid recommender combines content-based and collaborative filtering scores using a machine learning model to improve recommendation accuracy.

**Task 10: Feature-Based Explanations (For Content-Based Filtering) **

```
# Code for Task 10 - Feature-Based Explanation (Content-Based Filtering)

import numpy as np
import pandas as pd

def explain_recommendation(user_id, movie_id, top_features=3):

    # Build user profile
    user_profile = build_user_profile(user_id).flatten()

    # Get movie index
    movie_idx = movies[movies['movieId']==movie_id].index

    if len(movie_idx) == 0:
        return "Movie not found"

    movie_vector = tfidf_matrix[movie_idx].toarray().flatten()

    # Feature names (genres)
    feature_names = tfidf.get_feature_names_out()

    # Contribution = element-wise product
    contribution = user_profile * movie_vector

    # Get top contributing features
    top_indices = np.argsort(contribution)[::-1][:top_features]

    top_genres = [feature_names[i] for i in top_indices if contribution[i] > 0]

    if len(top_genres) == 0:
        return "No strong feature based explanation available."

    movie_title = movies[movies['movieId']==movie_id]['title'].values[0]

    explanation = f"{movie_title} was recommended because you like: " + ", ".join(top_genres)

    return explanation

# TEST EXPLANATION

# Get recommendations first
recs = recommend_for_user(1,5)

print("Recommendations:")
print(recs)

print("\nExplanations:\n")

for _, row in recs.iterrows():

    movie_title = row['title']
    movie_id = movies[movies['title']==movie_title]['movieId'].values[0]

    print(explain_recommendation(1, movie_id))
```

**Conclusion Task 10 : **Feature-based explanations are generated by identifying the genres that contribute most to the similarity between the user profile and the recommended movie. This improves transparency by showing why a particularrecommendation was made.

Task 11: Neighborhood-Based Explanations (For Collaborative Filtering)

```
# Code for Task 11 - Neighborhood-Based Explanation (Collaborative Filtering)

import numpy as np

def explain_cf_recommendation(user_id, movie_id, k=5):

    # Get top-K similar users
    similar_users = user_similarity[user_id].drop(user_id)
    similar_users = similar_users.sort_values(ascending=False).head(k)

    # Users who rated this movie
    user_ratings = user_item.loc[similar_users.index, movie_id]

    mask = user_ratings.notnull()

    if mask.sum() == 0:
        return "No similar users have rated this movie."

    sim_users = similar_users[mask]
    ratings = user_ratings[mask]

    # Build explanation
    explanation = "Recommended because similar users liked this movie:\n"

    for u, r in zip(sim_users.index, ratings):
        explanation += f"User {u} (similarity={sim_users[u]:.2f}) rated it {r}\n"

    return explanation



# Explanation of Test

# Get CF recommendations
recs = recommend_cf(1,5)

print("Recommendations:")
print(recs)

print("\nExplanations:\n")

for _, row in recs.iterrows():
    movie_id = row['movieId']
    print(f"{row['title']}:")
    print(explain_cf_recommendation(1, movie_id))
    print("-"*50)
```

Conclusion Task 11 : Neighborhood-based explanations are generated by identifying similar users who have rated a movie highly. The recommendation is explained by showing how these similar users influenced the prediction.

Task 13: Evaluating Explainability Compare how different explainability methods affect user understanding. Do explanations make recommendations clearer? Do explanations reveal biases in recommendations

```
# Code for Task 13 - Evaluating Explainability

import pandas as pd

# Collect explanations for one user
user_id = 1

cbf_recs = recommend_for_user(user_id, 3)
cf_recs = recommend_cf(user_id, 3)

results = []

# --- Content-Based Explanations ---
for _, row in cbf_recs.iterrows():
    movie_id = movies[movies['title']==row['title']]['movieId'].values[0]
    explanation = explain_recommendation(user_id, movie_id)

    results.append({
        "Method": "Content-Based",
        "Movie": row['title'],
        "Explanation": explanation,
        "Clarity": 4,        # subjective score (1–5)
        "Interpretability": 5
```

```
        })

    # --- Collaborative Filtering Explanations ---
    for _, row in cf_recs.iterrows():
        movie_id = row['movieId']
        explanation = explain_cf_recommendation(user_id, movie_id)

        results.append({
            "Method": "Collaborative Filtering",
            "Movie": row['title'],
            "Explanation": explanation,
            "Clarity": 3,
            "Interpretability": 4
        })

    df_explain = pd.DataFrame(results)

    print("Explainability Comparison:\n")
    print(df_explain)

    # Summary
    summary = df_explain.groupby("Method")[["Clarity","Interpretability"]].mean()

    print("\nAverage Scores:\n")
    print(summary)
```

Conclusion Q13 Content-based explanations tend to be more intuitive because they draw directly on movie attributes such as genres. By contrast, collaborative filtering explanations emphasize user similarity, though they are often harder to interpret. Offering explanations enhances transparency and helps users grasp the reasoning behind recommendations. At the same time, these explanations can expose biases, including the overemphasis on popular genres or dominant user groups.